No todo es Programación

La mayoría de las personas elegimos estudios por lo atrayente que resulta el nombre de esos estudios. En Formación Profesional hay muchísima variedad de títulos: Mantenimiento de Helicópteros, Anatomía Patológica, Energías Renovables, Dirección de Cocina, Desarrollo de Aplicaciones Web,...

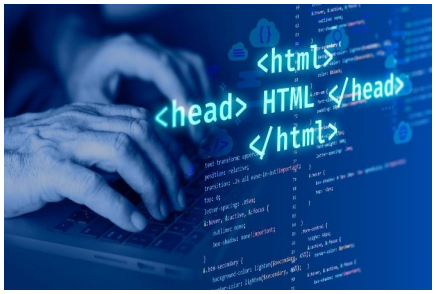
El título sintetiza en una frase en qué consiste la formación pero los módulos que componen el ciclo están diseñados para proporcionar al alumnado una formación integral, tanto teórica como práctica, que responda a las necesidades reales del sector profesional al que está orientado. Cada **módulo** cumple una función específica en el desarrollo de competencias clave, desde los conocimientos técnicos hasta las habilidades transversales necesarias.

Además, un entorno profesional está en constante evolución y debe actualizar los contenidos de sus módulos y, así, garantizar que los futuros profesionales estén preparados para afrontar los retos del sector, incorporando las últimas tecnologías y buenas prácticas.



1. Lenguajes de Marcas

El origen de las páginas web era el acceso y la organización de documentos de divulgación científica. Los documentos incluirían referencias a otros documentos relacionados con el tema a investigar, de esa manera, se podría acceder a muchas más información sobre el mismo tema a través de enlaces, que conectaban unos documentos con otros.



Para dar formato a estos documentos se desarrollo un lenguaje de marcado llamado HTML (HyperText Markup Language) pensado, principalmente, para diseño de páginas web.

La característica principal de estos lenguajes son las etiquetas (**marcas**) utilizadas para marcar el inicio y el final de un elemento, en particular, los componentes de una web.

La componente más propia de las páginas web es el **enlace**, marcado por las etiquetas <a> y . Además la etiqueta de apertura tiene un argumento llamado *href* que indica cuál es en enlace al que apunta ese elemento.

Entrada

Nuestro ejercicio comienza por un número *n* indicado cuántos casos de prueba tenemos. Cada caso de prueba está formado por dos cadena de texto sin espacios. La primera representa una *url* que representa un enlace a un elemento web. La segunda representa una sola *palabra* y que en una página web representa el enlace al elemento web.

Salida

Para cada caso el programa transformará los datos de entrada en un enlace en formato HTML.
La salida **no se mostrará por pantalla.**
Cada caso se guardará en una línea de un fichero llamado *enlaces.html*

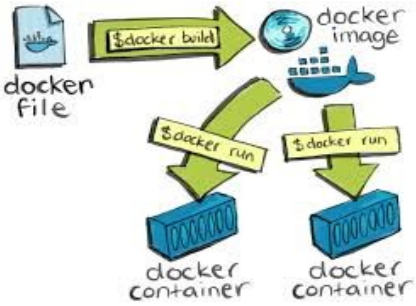
Entrada de ejemplo

```
3
www.google.com buscador
ieslasenia.org la_senia
openai.com/chatgpt chatGPT
```

Salida de ejemplo (**Se guarda en el fichero enlaces.html**)

```
<a href="http://www.google.com">buscador</a>
<a href="http://ieslasenia.org">la_senia</a>
<a href="http://openai.com/chatgpt">chatGPT</a>
```

2. Sistemas y Entornos



Un **dockerfile** es un **archivo de texto** que contiene un conjunto de instrucciones que **Docker** sigue de manera secuencial para construir una imagen de un contenedor. Es como un **script** que automatiza la configuración de un entorno.

Un ejemplo de un DockerFile puede ser:

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y python3
COPY app.py /app/app.py
CMD ["python3", "/app/app.py"]
```

Los ficheros dockerfile contienen dos tipos de líneas: Las que empiezan por el símbolo # que indican que son comentarios; y los que empiezan por una palabra en mayúsculas que representa una acción (`FROM`, `RUN`, `ENV`, `COPY`, `CMD`, `WORKDIR`, `USER`,...) para construir una imagen.

Vamos a crear un programa en Java que **tome como argumento** un fichero dockerfile y **muestre por pantalla** un listado de lo que contiene.

Entrada de ejemplo (fichero dockerfile)

```
# Imagen base con Python
FROM python:3.11-slim
# Crear un usuario y un grupo
RUN addgroup --system appgroup
RUN adduser --system --ingroup appgroup appuser
# Crear directorio de trabajo y dar permisos al nuevo usuario
WORKDIR /app
RUN chown appuser:appgroup /app
# Copiar archivos de la aplicación
COPY requirements.txt .
COPY config.md .
RUN pip install --no-cache-dir -r requirements.txt
# Cambiar propietario de todos los archivos copiados
RUN chown -R appuser:appgroup /app
# Cambiar a usuario no root
USER appuser
# Exponer puerto
EXPOSE 8000
# Ejecutar la app
CMD ["python", "app.py"]
```

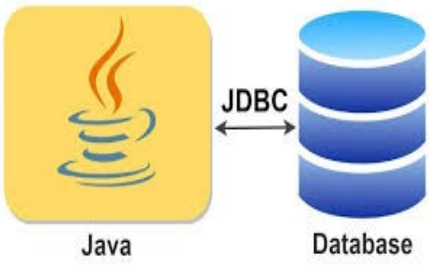
Salida de ejemplo

```
Comentarios 8
FROM 1
RUN 5
WORKDIR 1
COPY 2
USER 1
EXPOSE 1
CMD 1
```

3. Bases de Datos

Los últimos **resultados de aprendizaje** del módulo de **Programación** están relacionados con el uso de Bases de Datos y la gestión de la información almacenada en ellas.Por ello, era de esperar que habría un ejercicio que usara las bases de datos para mantener la persistencia de la información.

Si por algo se ha caracterizado este curso ha sido por las situaciones imprevistas que hemos tenido. Pese a que la enseñanza es presencial hemos tenido clases online y, sobre todo, que la NO asistencia a clase no ha supuesto anulación de los estudios.



Normalmente es muy importante el **control de la asistencia** ya que la FP es enseñanza financiada por los FSE y la asistencia es obligatoria.

Vamos a crear un programa de gestión de faltas de asistencia y para ello nos apoyaremos en la base de datos **FaltasDAW** (fichero faltasDAW.sql).

Crearemos la **clase Alumno** que tendrá lo necesario para gestionar la tabla alumnos.
Crearemos la **clase Sesion** con lo necesario para gestionar la tabla sesiones.
Nota: crear el método *toString()* para usarlo posteriormente según el formato que se explica

Al iniciar el programa volcamos los datos de la base de datos: creamos una **lista de Alumnos** y una **lista de Sesiones** que mantendremos y utilizaremos para gestionar las faltas (opción 3).

Creamos un menú para el programa:

1. Alta Alumno/a
2. Alta Nueva Sesion
3. Introducir falta
4. Salir

La **opción 1** permite dar de alta un Alumno/a. Pedirá los datos de *NIA*, *nombre* y *edad* y los guardará en la BD.

La **opción 2** permite dar de alta una nueva Sesion. Pedirá los datos de mes (1-12), día (1-31) y sesión (1-13) y compondrá el *IdSesion* (gestionar los posibles errores con excepciones). También pedirá el *modulo*. Con todo lo guardará en la BD.

La **opción 3** permite introducir una falta.

1. Se mostrará un listado de los nombres de los alumnos en formato:

   <NumAlumno> - <nombre> [método toString()]

   Elegiremos uno usando el NumAlumno
2. Se mostrará un listado de las sesiones en formato:

   <IdSesion> - <modulo> [método toString()]

   Elegiremos uno usando el IdSesion

3. Se pide el tipo de falta: 1 para falta y 2 para retraso
4. Se pregunta si está justificada: FALSE (0) o TRUE (1).
5. Cuando se tenga todo se inserta en la BD. (Importante que sean correctos NumAlumno y IdSesion)

La **opción 4** finaliza el programa.

Solo finaliza el programa con esa opción cualquier otra opción mostrará nuevamente el menú.

Crea las clases, los atributos y los métodos indicados.

Crea el programa para gestionar las faltas de asistencia

Ejemplo opcion3

```
1. Alta Alumno/a
2. Alta Nueva Sesion
3. Introducir falta
4. Salir
Elige opcion: 3
1 - Andrea García Ruiz
2 - Luisa Bora Lima
3 - Alberto Capelo Tazo
4 - Carmen Tira Grande
Elige opcion: 2
050710 - IPE
050711 - PRG
050712 - PRG
050713 - PRG
050808 - PI
050809 - EDE
050908 - BDA
Elige opcion: 050713
Elige: 1 (Falta) | 2 (Retraso)
1
Justificante? 0 (NO) | 1 (SI)
0
```

```
select * from faltas;
```

Results

NumAlumno	IdSesion	Tipo	Just
2	050713	1	0